

Complexidade de Algoritmos

Conceitos Básicos

Prof. Leandro Israel Pinto

❖ Complexidade e Desempenho;

- ❖ A complexidade é o esforço computacional para executá-lo sobre uma entrada de tamanho n ;

- ❖ Existem vários aspectos a considerar da complexidade de um algoritmo.
 - ❖ Uma é a medida empírica;
 - ❖ Podemos medir experimentalmente o tempo de um algoritmo em uma máquina;
 - ❖ Porém, é fortemente dependente;
 - ❖ Depende do programa e da máquina;
- ❖ Uma alternativa é fazer uma análise matemática do algoritmo;
 - ❖ É independente da implementação;
 - ❖ Permite antecipar o cálculo da complexidade para a fase de projeto do algoritmo;
 - ❖ Notação Big O

- ❖ A complexidade não pode ser descrita simplesmente por um número;
 - ❖ Pois a quantidade de trabalho depende da entrada;

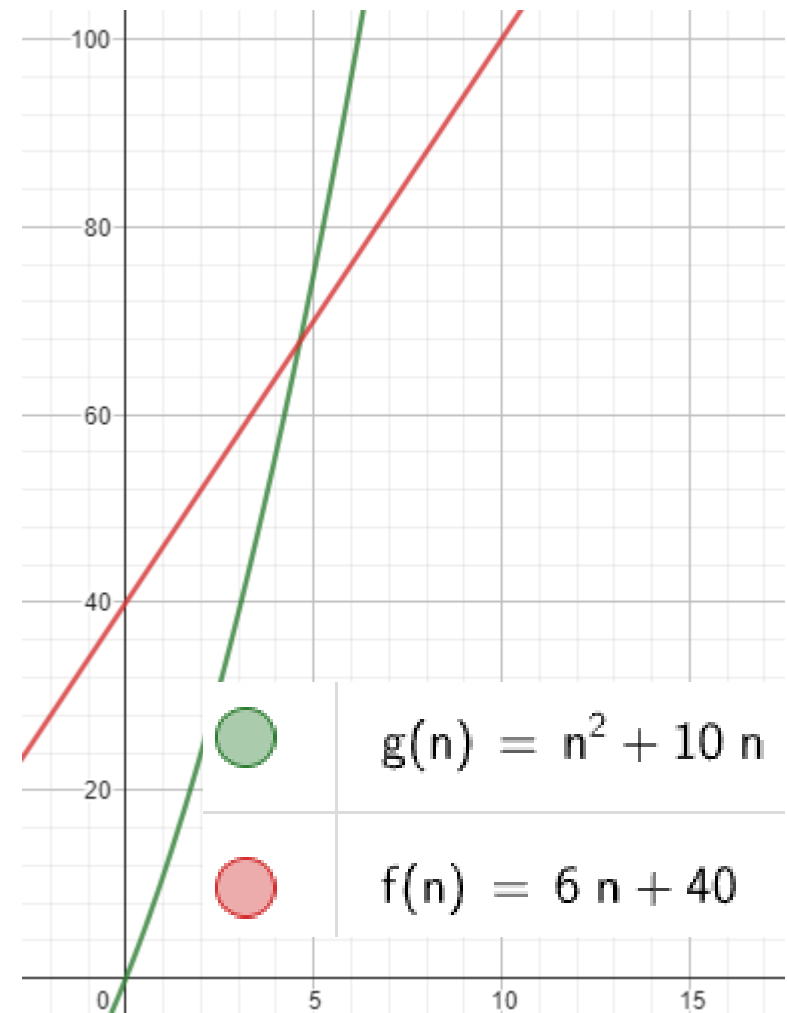
- ❖ O tempo requerido por um algoritmo pode ser medido pelo número de execuções de algumas operações;
- ❖ Costuma-se escolher uma **operação fundamental**;
 - ❖ O número de vezes que ela é executada expressa a quantidade de trabalho do algoritmo;
- ❖ Exemplo: Num algoritmo de ordenação, a operação fundamental seria a comparação entre elementos;
- ❖ Pode ser necessária mais de uma;

Pior caso, melhor caso, caso médio

- ❖ O número de operações pode depender da entrada;
 - ❖ Ex.: Entrada parcialmente ordenada para um algoritmo de ordenação;
- ❖ Em geral, nos interessa o **pior caso**.
- ❖ Também podemos analisar o melhor caso e o caso médio;
- ❖ Exemplo: Numa busca em um vetor de n elementos, qual o melhor, pior e médio?

Comportamento assintótico

- ❖ Dependendo do valor de n , um algoritmo poderá ter mais operações que outro;
- ❖ Um caso de interesse é quando n tem um valor grande;
- ❖ Termos inferiores e constantes multiplicativas contribuem pouco;
- ❖ O importante é $g(n) = n^2$ e $f(n) = n$;



- ❖ $n^2 + 10n$ tem o mesmo crescimento assintótico que n^2 ;
- ❖ Uma cota assintótica superior (CAS) é uma função que cresce mais rapidamente do que outra: permanece acima a partir de certo ponto.

$$(\exists n_0 \in \mathbb{N})(\forall n \geq n_0): f(n) \leq g(n)$$

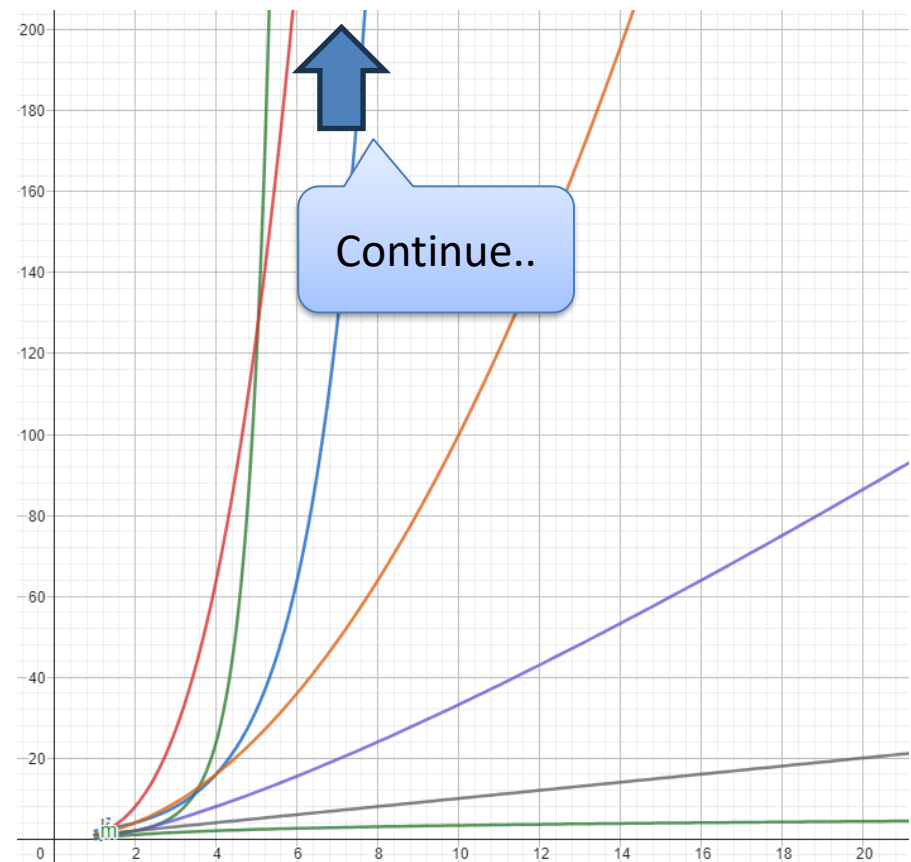


Para n suficientemente grande, $g(n)$ domina $f(n)$

- ❖ Descreve o pior caso de tempo de execução do algoritmo, em termos de uma função que representa o crescimento assintótico;
- ❖ Define uma cota assintótica superior a menos de constantes;
- ❖ Exemplo: a função $g(n) = n^2$ cresce mais rapidamente do que a função $f(n) = n$
 - ❖ Dizemos que a linear $f(n)$ é $O(g(n))$
 - ❖ $f(n)$ “é pequena para” $g(n)$
 - ❖ $f(n) = O(g(n))$

❖ <https://www.geogebra.org/calculator>

	$f(x) = x!$
	$g(x) = 2^x$
	$h(x) = x^3$
	$i(x) = x^2$
	$j(x) = x \log_2(x)$
	$k(x) = x$
	$m(x) = \log_2(x)$



❖ Complexidade constante

```
1  int a, b, c;           //O(1)
2  a = 100;               //O(1)
3  b = a * 2 * b          //O(1)
4  if(a < b)               //O(1)
5      c = a;              //O(1)
```

$$O(1) + O(1) = O(1)$$

```
1  for(i = 0; i < n; i++) {  
2      //O(1);  
3  }  
4  
5  for(i = 0; i < n; i++) {  
6      //O(n);  
7  }  
8  
9  for(i = 0; i < n; i++) {  
10     //O(n^2);  
11 }
```

$$O(n)$$

$$n * n = O(n^2)$$

$$O(n^3)$$

```
1  for (i = 0; i < n; i++) {  
2      //O(m) ;  
3  }
```

$$n.m = O(nm)$$

```
1  for(i = 0; i < n; i++) {  
2      //O(n);  
3  }  
4  for(i = 0; i < n; i++) {  
5      //O(n^2);  
6  }
```

$O(n^2)$

$O(n^3)$

$$n^2 + n^3 = O(n^3)$$

- ❖ Qual a complexidade do algoritmo abaixo? Reescreva-o de forma que tenha uma complexidade menor.
 - ❖ Expresse a quantidade de operações em relação a n ;
 - ❖ Crie uma função número de operações X tempo;
 - ❖ Encontre O: A classe de funções que melhor descreve o crescimento assintótico. Isso é a complexidade do algoritmo.

```
1 def processamento_quadratico(lista):  
2     resultado = 0  
3     n = len(lista)  
4     for i in range(n):  
5         for j in range(n):  
6             resultado += lista[i] * lista[j]  
7     return resultado
```


$i = 0 \quad j = 0..n-1 \quad n$

$i = 1 \quad j = 0..n-1 \quad n$

...

$i = n \quad j = 0..n-1 \quad n$

Teremos que n é somado n vezes:

$$(n + n + \dots + n) = n * n = n^2$$

$$O(n^2)$$

- ❖ Para poder simplificar o algoritmo, entenda sua operação e os dados!
- ❖ Supondo que os dados são o vetor $[a, b]$
- ❖ A execução faria:

$$a.a + a.b + b.a + b.b$$

$$a(a+b) + b(a+b)$$

- ❖ Como fica um algoritmo com menor complexidade?

```
int pesquisa(Estrutura *v, int n, int chave){
    int i;
    i = 0;
    while(i < n){
        if(v[i].chave == chave){
            return i;
        }
        i++;
    }
    return -1;
}
```

- ❖ Qual a probabilidade do elemento estar em cada uma das posições?

- ❖ Assumindo distribuição uniforme:
- ❖ O número de comparações é o somatório

$$\sum_{i=1}^n c_i p_i$$

- ❖ c_i : Número de comparações para encontrar o elemento i ;
- ❖ p_i : Probabilidade do elemento estar na posição i

$$1.\frac{1}{n} + 2.\frac{1}{n} + 3.\frac{1}{n} + \dots + (n-1).\frac{1}{n} + n.\frac{1}{n}$$
$$\frac{1}{n} \cdot (1 + 2 + \dots + (n-1) + n)$$

$$\begin{aligned}
 &1 \cdot \frac{1}{n} + 2 \cdot \frac{1}{n} + 3 \cdot \frac{1}{n} + \dots + (n-1) \cdot \frac{1}{n} + n \cdot \frac{1}{n} \\
 &\quad \frac{1}{n} \cdot (1 + 2 + \dots + (n-1) + n) \\
 &\quad \frac{1}{n} \cdot \frac{n \cdot (n+1)}{2}
 \end{aligned}$$

$$O(n)$$